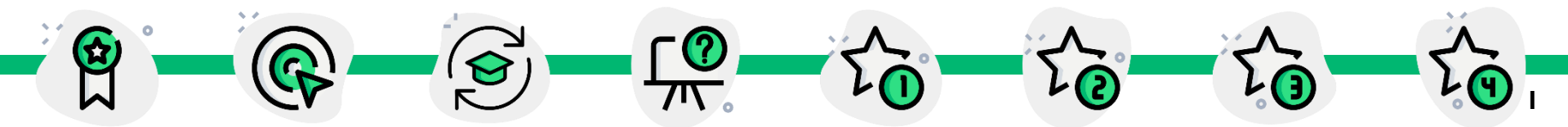


「2026년도 1학기 한국성서대학교 AI융합학부」

알고리즘

2026.06.15

15주차



트리



09-1 트리란?

트리 정의하기 - (1)

■ 트리 관련 용어 - (1)

- 트리를 구성하는 요소는 노드(node)와 가지(edge)
- 각각의 노드는 가지를 통해 다른 노드와 연결되어 있음

■ 루트 root

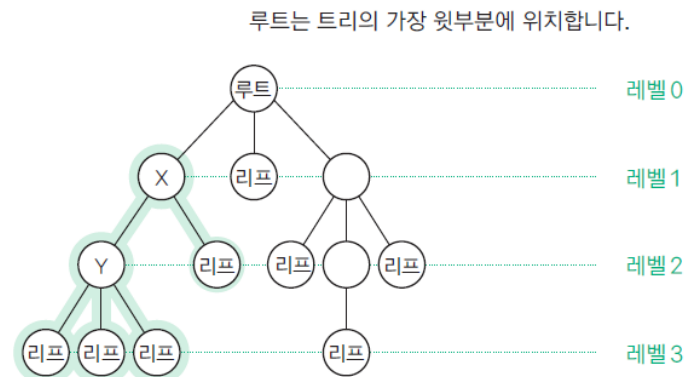
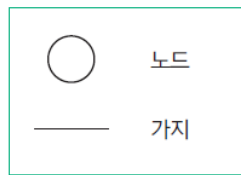
- 트리의 가장 윗부분에 위치하는 노드
- 하나의 트리에는 하나의 루트가 있음
- 그림을 거꾸로 보면 나무 모양과 비슷함

■ 리프 leaf

- 트리의 가장 아랫부분에 위치하는 노드
- '가장 아래에 위치한다'라는 말은 물리적으로 가장 아랫부분에 위치한다는 의미가 아니라 더 이상 뺄어나갈 수 없는 마지막에 노드가 위치한다는 의미

■ 안쪽 노드

- 루트를 포함하여 리프를 제외한 노드



초록색으로 표시한 부분은 서브 트리라고 부릅니다.
이때 서브 트리의 X는 다시 서브 트리의 루트 역할을 합니다.

[그림 9-1] 트리

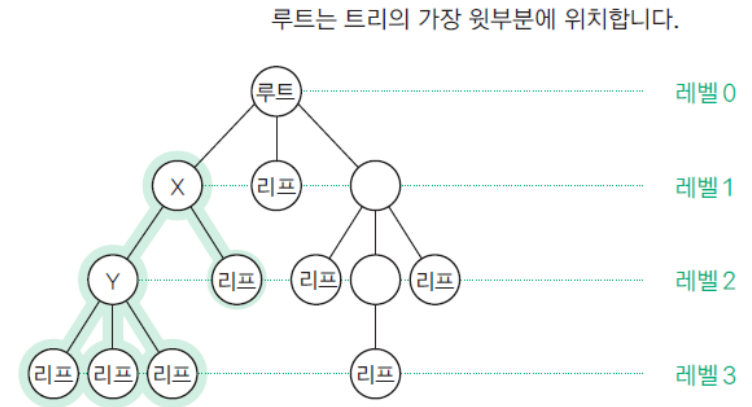
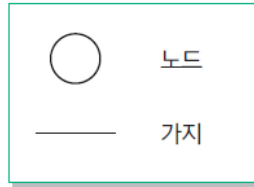


09-1 트리란?

트리 정의하기 - (2)

■ 트리 관련 용어 - (2)

- 자식^{child}
 - 어떤 노드로부터 가지로 연결된 아래쪽 노드
 - 노드는 자식을 여러 개 가질 수 있음
- 부모^{parent}
 - 어떤 노드에서 가지로 연결된 위쪽 노드
 - 노드는 1개의 부모를 가짐
- 형제^{sibling}
 - 같은 부모를 가지는 노드
- 조상^{ancestor}
 - 어떤 노드에서 가지로 연결된 위쪽 노드 모두
- 자손^{descendant}
 - 어떤 노드에서 가지로 연결된 아래쪽 노드 모두



초록색으로 표시한 부분은 서브 트리라고 부릅니다.
이때 서브 트리의 X는 다시 서브 트리의 루트 역할을 합니다.

[그림 9-1] 트리



09-1 트리란?

트리 정의하기 - (3)

■ 트리 관련 용어 - (3)

■ 레벨^{level}

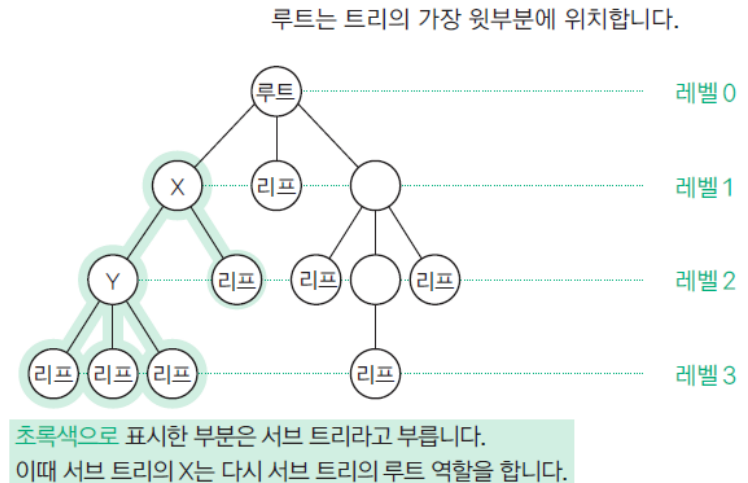
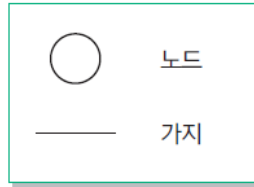
- 루트로부터 얼마나 떨어져 있는지에 대한 값
- 루트의 레벨은 0이고 루트로부터 가지가 하나씩 아래로 뻗어나갈 때마다 레벨이 1씩 늘어남

■ 차수^{degree}

- 노드가 갖는 자식의 수
- 모든 노드의 차수가 n 이하인 트리를 n 진 트리라고 함

■ 높이^{height}

- 루트부터 가장 멀리 떨어진 리프까지의 거리(리프 레벨의 최댓값)



[그림 9-1] 트리



09-1 트리란?

트리 정의하기 - (4)

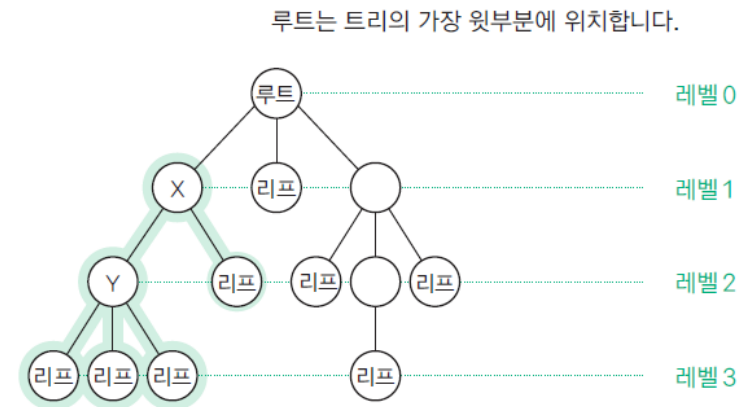
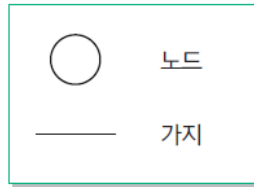
■ 트리 관련 용어 - (4)

■ 서브 트리_{subtree}

- 트리 안에서 다시 어떤 노드를 루트로 정하고 그 자손으로 이루어진 트리

■ 널 트리_{null tree}

- 노드, 가지가 없는 트리



초록색으로 표시한 부분은 서브 트리라고 부릅니다.
이때 서브 트리의 X는 다시 서브 트리의 루트 역할을 합니다.

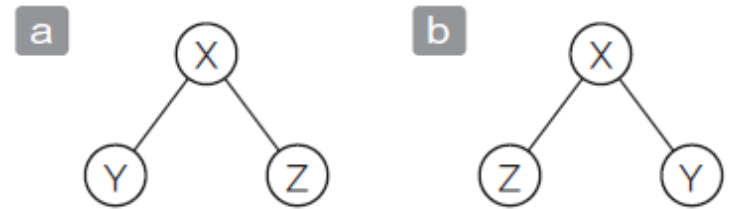
[그림 9-1] 트리



09-1 트리란?

순서 트리와 무순서 트리 정의하기

- 순서 트리와 무순서 트리
 - 형제 노드의 순서가 있는지 없는지에 따라 트리를 두 종류로 분류
 - 형제 노드의 순서를 따지면 순서 트리(ordered tree), 따지지 않으면 무순서 트리(unordered tree)라고 함



두 트리는 다른 순서 트리이면서
같은 무순서 트리라고 할 수 있습니다.

[그림 9-2] 순서 트리와 무순서 트리

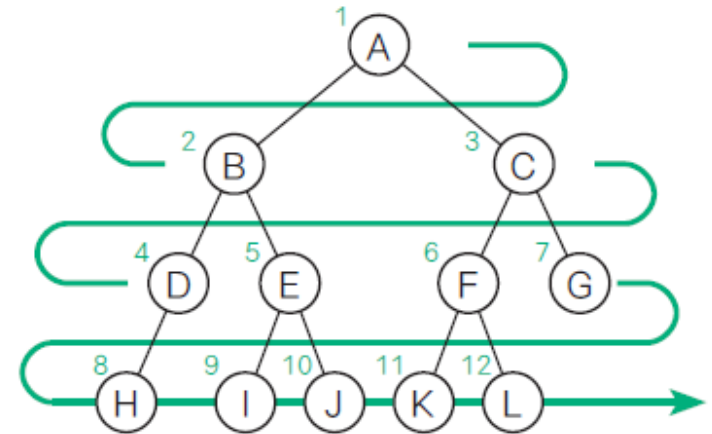


09-1 트리란?

순서 트리의 탐색 방법 알아보기 - (1)

■ 너비 우선 탐색 breadth-first Search

- 낮은 레벨에서 시작해 왼쪽에서 오른쪽 방향으로 검색하고 한 레벨에서의 검색이 끝나면 다음 레벨로 내려감
- 검색 순서: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G \rightarrow H \rightarrow I \rightarrow J \rightarrow K \rightarrow L$



[그림 9-3] 너비 우선 탐색

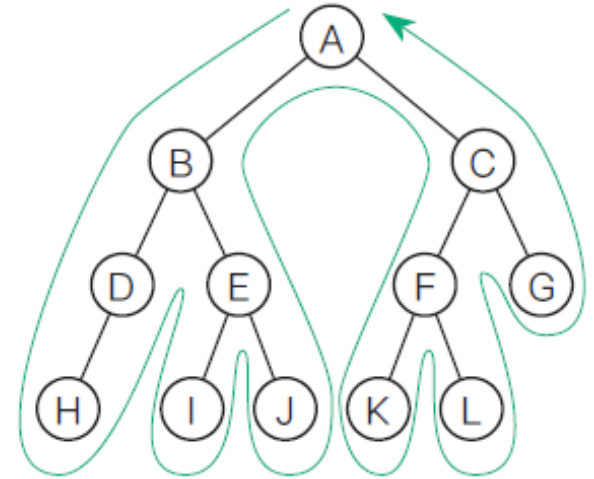


09-1 트리란?

순서 트리의 탐색 방법 알아보기 - (2)

■ 깊이 우선 탐색 depth-first Search

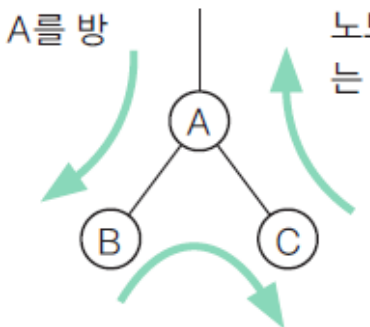
- 리프까지 내려가면서 검색하는 것을 우선순위로 하는 탐색 방법
- 리프에 도달해 더 이상 검색을 진행할 곳이 없는 경우에는 부모에게 돌아감
- 그런 다음 다시 자식 노드로 내려감
- 그림 9-5는 노드 A를 몇 번 지나갔는지를 나타낸 것
- 두 자식 가운데 한쪽(또는 양쪽)이 없으면 노드를 지나가는 횟수가 줄어들겠지만 노드를 지나가는 횟값은 3회



[그림 9-4] 깊이 우선 탐색

① 출발하려고 하는
경우에 노드 A를 방
문하는 경우

③ 되돌아오면서
노드 A를 방문하
는 경우



② 지나가면서 노드 A를
방문하는 경우

[그림 9-5] 깊이 우선 탐색에서 가능한 방문 종류



09-1 트리란?

순서 트리의 탐색 방법 알아보기 - (3)

■ 깊이 우선 탐색에서 가능한 노드 방문 종류

■ 전위 순회|Preorder

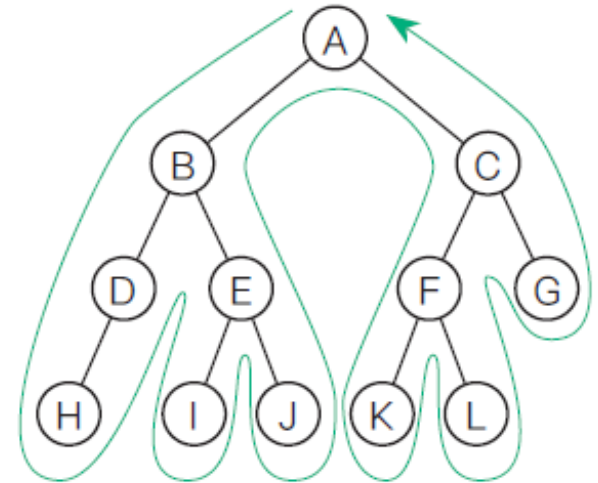
- 노드 방문 → 왼쪽 자식 → 오른쪽 자식
- $A \rightarrow B \rightarrow D \rightarrow H \rightarrow E \rightarrow I \rightarrow J \rightarrow C \rightarrow F \rightarrow K \rightarrow L \rightarrow G$

■ 중위 순회|Inorder

- 왼쪽 자식 → 노드 방문 → 오른쪽 자식
- $H \rightarrow D \rightarrow B \rightarrow I \rightarrow E \rightarrow J \rightarrow A \rightarrow K \rightarrow F \rightarrow L \rightarrow C \rightarrow G$

■ 후위 순회|Postorder

- 왼쪽 자식 → 오른쪽 자식 → (돌아와) 노드 방문
- $H \rightarrow D \rightarrow I \rightarrow J \rightarrow E \rightarrow B \rightarrow K \rightarrow L \rightarrow F \rightarrow G \rightarrow C \rightarrow A$



[그림 9-4] 깊이 우선 탐색

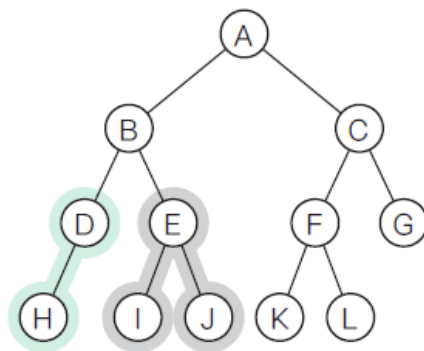


09-2 이진트리와 이진검색트리

이진트리 정의하기

■ 이진트리 binary tree

- 노드가 왼쪽 자식과 오른쪽 자식을 갖는 트리
- 노드의 자식은 2명 이하만 유지해야 함
- 왼쪽 자식과 오른쪽 자식을 구분함
- 왼쪽 자식을 다시 루트로 하는 서브 트리를 왼쪽 서브 트리(left subtree), 오른쪽 자식을 다시 루트로 하는 서브 트리를 오른쪽 서브 트리(right subtree)라고 함



노드 B의 왼쪽 서브 트리

노드 B의 오른쪽 서브 트리

[그림 9-6] 이진트리

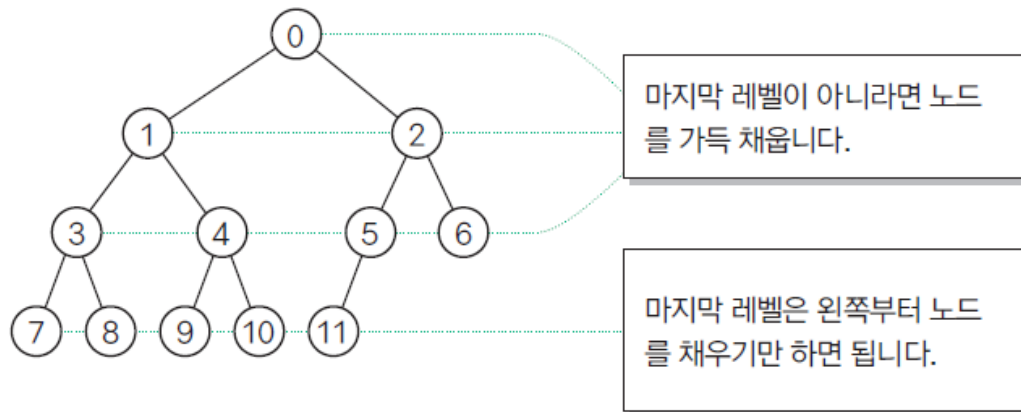


09-2 이진트리와 이진검색트리

완전이진트리 정의하기

■ 완전이진트리 complete binary tree

- 루트부터 노드가 채워져 있으면서 같은 레벨에서는 왼쪽에서 오른쪽으로 노드가 채워져 있는 이진트리
 - 1. 마지막 레벨을 제외한 레벨은 노드를 가득 채움
 - 2. 마지막 레벨은 왼쪽부터 오른쪽 방향으로 노드를 채우되 반드시 끝까지 채울 필요는 없음
- 높이가 k 인 완전이진트리가 가질 수 있는 노드의 최댓값은 $2^{k+1}-1$ 개
- 따라서 n 개의 노드를 저장할 수 있는 완전이진트리의 높이는 $\log n$
- 완전이진트리에서 너비 우선 탐색을 하며 각 노드에 $0, 1, 2, \dots$ 값을 주면 배열에 저장하는 인덱스와 일대일로 대응



[그림 9-7] 완전이진트리



09-2 이진트리와 이진검색트리

이진검색트리 살펴보기

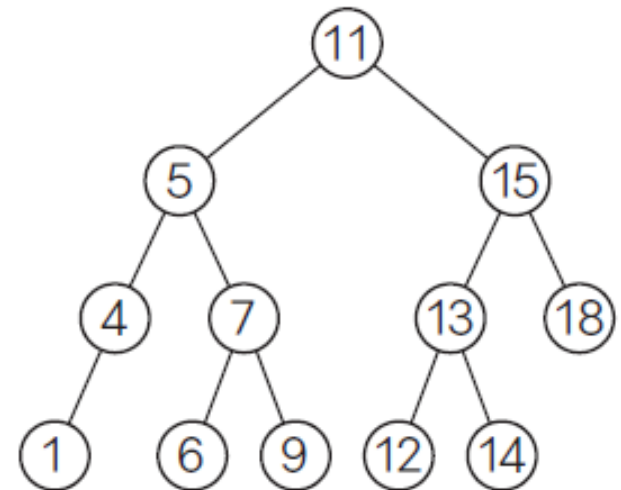
■ 이진검색트리 binary Search tree

■ 이진트리가 다음의 조건을 만족

- 1. 어떤 노드 N을 기준으로 왼쪽 서브 트리 노드의 모든 키값은 노드 N의 키값보다 작아야 함
- 2. 오른쪽 서브 트리 노드의 키값은 노드 N의 키값보다 커야 함
- 3. 같은 키값을 갖는 노드는 없음

■ 이진검색트리의 특징

- 구조가 단순함
- 중위 순회를 하면 키값의 오름차순으로 노드를 얻을 수 있음
- 이진검색과 비슷한 방식으로 빠르게 검색할 수 있음
- 노드를 삽입하기 쉬움



[그림 9-8] 이진검색트리의 구현



09-2 이진트리와 이진검색트리

이진검색트리 만들기 - (1)

■ 실습 9-1

■ 이진검색트리 프로그램

Do it! 실습 9-1

• 완성 파일 chap09/BinTree.h

```
01 // 이진검색트리 프로그램(헤더)
02 #ifndef __BinTree
03 #define __BinTree
04
05 #include "Member.h" — 실습 10-1에서 작성
06
07 /*--- 노드 ---*/
08 typedef struct __bnode {
09     Member data;           // 데이터
10     struct __bnode *left;  // 왼쪽 자식 노드에 대한 포인터
11     struct __bnode *right; // 오른쪽 자식 노드에 대한 포인터
12 } BinNode;
13
14 /*--- 검색 ---*/
15 BinNode *Search(BinNode *p, const Member *x);
16
```

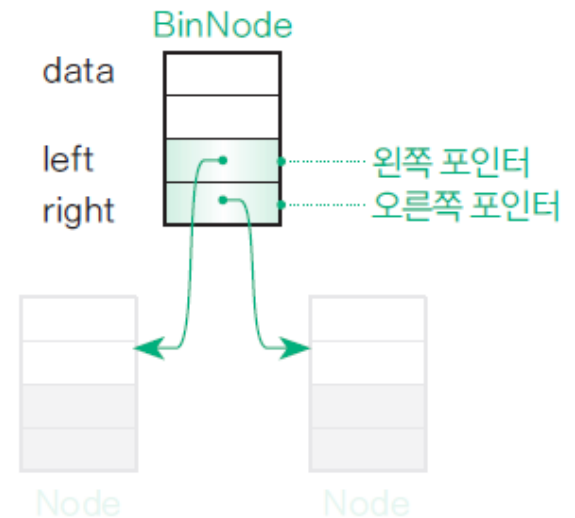
```
17 /*--- 노드 삽입 ---*/
18 BinNode *Add(BinNode *p, const Member *x);
19
20 /*--- 노드 삭제 ---*/
21 int Remove(BinNode **root, const Member *x);
22
23 /*--- 모든 노드를 출력 ---*/
24 void PrintTree(const BinNode *p);
25
26 /*--- 모든 노드를 삭제 ---*/
27 void FreeTree(BinNode *p);
28 #endif
```



09-2 이진트리와 이진검색트리

이진검색트리 만들기 - (2)

- 노드를 표현하는 구조체 BinNode
 - 이진검색트리의 개별 노드는 자기 참조형 구조체인 BinNode
 - data ... 회원 데이터
 - left ... 왼쪽 포인터(왼쪽 자식 노드에 대한 포인터)
 - right ... 오른쪽 포인터(오른쪽 자식 노드에 대한 포인터)
 - left와 right 멤버가 좌우의 자식 노드에 대한 포인터
 - 가리킬 자식 노드가 없을 때는 NULL이 됨



[그림 9-9] 구조체 BinNode



09-2 이진트리와 이진검색트리

이진검색트리 만들기 - (3)

- 노드를 생성하는 **AllocBinNode** 함수
 - BinNode형 객체를 만드는 함수
- 노드 멤버값을 설정하는 **SetBinNode** 함수
 - BinNode형 객체의 3개 멤버에 값을 설정하는 함수로, 첫 번째 매개변수 n이 가리키는 BinNode형 객체에 대해 멤버값을 설정
 - n이 가리키는 객체 멤버인 data, left, right에 두 번째 매개변수 x가 가리키는 객체값 *x와 세 번째, 네 번째 매개변수의 포인터값 left, right를 각각 대입
- 비어 있는 상태의 이진검색트리 만들기
 - Initialize 함수 대신 그림 9-10과 같이 루트 노드를 가리키는 BinNode *형 객체를 하나 준비하고 널 값을 대입
 - 루트 노드를 가리키는 포인터는 이진검색트리를 사용하는 실습 9-3의 main 함수에서 미리 선언



09-2 이진트리와 이진검색트리

• 완성 파일 chap09/BinTree.h

Do it! 실습 9-2[A]

```
01 // 이진검색트리 프로그램(소스)
02 #include <stdio.h>
03 #include <stdlib.h>
04 #include "Member.h"
05 #include "BinTree.h"
06
07 /*--- 노드를 동적으로 할당 ---*/
08 static BinNode *AllocBinNode (void)
09 {
10     return calloc(1, sizeof(BinNode));
11 }
12
13 /*--- 노드 멤버값 설정 ---*/
14 static void SetBinNode(BinNode *n, const Member *x, const BinNode *left, const BinNode *right)
15 {
16     n->data = *x;           // 데이터
17     n->left = left;         // 왼쪽 포인터
18     n->right = right;       // 오른쪽 포인터
19 }
20
```

(실습 9-2[B]에서 계속)

노드를 가리키는 상태가 아닙니다.

NULL

root

[그림 9-10] 빈 상태의 이진검색트리



09-2 이진트리와 이진검색트리

이진검색트리 만들기 - (4)

■ 키값으로 검색하는 Search 함수 - (1)

■ a 검색에 성공한 경우

- 1 루트를 선택(5): 3은 5보다 작기 때문에 왼쪽으로 검색을 진행
- 2 선택한 노드는 2: 3은 2보다 크기 때문에 오른쪽으로 검색을 진행
- 3 선택한 노드는 4: 3은 4보다 작기 때문에 왼쪽으로 검색을 진행
- 4 3에 도착해 검색에 성공

■ b 검색에 실패하는 경우

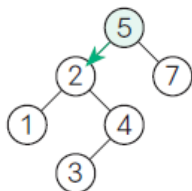
- 루트를 선택(5): 8은 5보다 크기 때문에 오른쪽으로 검색을 진행
- 7을 선택: 7은 리프이고 오른쪽 자식 노드가 없으므로 더 이상 검색할 수 없기 때문에 검색에 실패



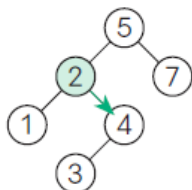
09-2 이진트리와 이진검색트리

a 3을 검색합니다(검색 성공).

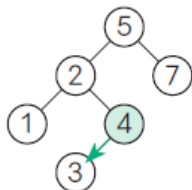
1 5를 선택합니다. 3은 5보다 작기 때문에 왼쪽으로 검색을 진행합니다.



2 2를 선택합니다. 3은 2보다 크기 때문에 오른쪽으로 검색을 진행합니다.

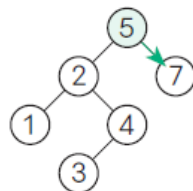


3 4를 선택합니다. 3은 4보다 작기 때문에 왼쪽으로 검색을 진행합니다.

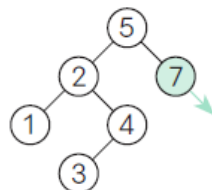


b 8을 검색합니다(검색 실패).

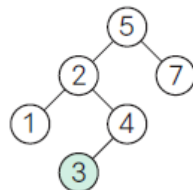
1 5를 선택합니다. 8은 5보다 크기 때문에 오른쪽으로 검색을 진행합니다.



2 7을 선택합니다. 8은 7보다 크기 때문에 오른쪽으로 검색을 진행합니다. 하지만 오른쪽에는 자식이 없어 검색에 실패합니다.



4 3을 찾았습니다. 검색에 성공합니다.





09-2 이진트리와 이진검색트리

이진검색트리 만들기 - (5)

■ 키값으로 검색하는 Search 함수 - (2)

- 첫 번째 매개변수 p를 루트로 하는 이진검색트리에서 검색을 수행
- 호출된 Search 함수는 x가 가리키는 구조체 Member형 객체와 같은 키값을 갖는 노드를 검색
- 검색에 성공하면 해당 노드에 대한 포인터를 반환

Do it! 실습 9-2[B]

```
01  /*--- 검색 ---*/
02  BinNode *Search (BinNode *p, const Member *x)
03  {
04      int cond;
05      if(p == NULL)
06          return NULL;          // 검색 실패
07      else if((cond = MemberNoCmp(x, &p->data)) == 0)
08          return p;              // 검색 성공
09      else if(cond < 0)
10          Search(p->left, x);    // 왼쪽 서브 트리에서 검색
11      else
12          Search(p->right, x);   // 오른쪽 서브 트리에서 검색
13  }
14
```

• 완성 파일 chap09/BinTree.c

1. 루트부터 선택하여 검색을 진행합니다. 여기서 선택하는 노드를 p라고 합니다.
2. p가 NULL이면 검색에 실패합니다.
3. 검색하는 값 key와 선택한 노드 p의 키값을 비교하여
 - 값이 같으면 검색에 성공(검색 종료)합니다.
 - key가 작으면 선택한 노드에 왼쪽 자식 노드를 대입합니다(왼쪽으로 검색 진행).
 - key가 크면 선택한 노드에 오른쪽 자식 노드를 대입합니다(오른쪽으로 검색 진행).
4. 2번 과정으로 되돌아갑니다.

```
/*--- 회원 번호 비교 함수 ---*/
int MemberNoCmp(const Member *x, const Member *y)
{
    return x->no < y->no ? -1 : x->no > y->no ? 1 : 0;
}
```

(실습 9-2[C]에서 계속)

- x->no < y->no 이면 -1
- x->no > y->no 이면 1
- 같으면 0



09-2 이진트리와 이진검색트리

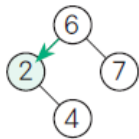
이진검색트리 만들기 - (6)

■ 노드를 삽입하는 Add 함수 - (1)

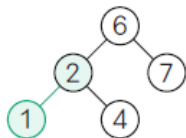
- 노드를 삽입할 때 주의해야 할 점은 노드를 삽입한 다음에 트리의 형태가 이진검색트리의 조건을 유지해야 한다는 점
- 따라서 노드를 삽입할 때는 먼저 삽입하기에 알맞은 위치를 찾아내야 함
- 이때 삽입할 노드의 키와 같은 값을 가지는 노드가 이미 있다면 노드를 삽입해서는 안 됨

a 1을 삽입하는 과정

검색하는 방법과 마찬가지로 삽입할 위치를 찾습니다. 추가할 값인 1은 2보다 작고, 2의 왼쪽 자식 노드는 비어 있으므로 해당 위치를 삽입 위치로 선택합니다.

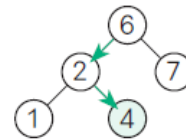


2의 왼쪽 자식 노드로 1을 삽입합니다.

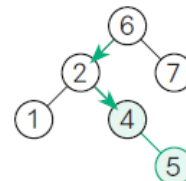


b 5를 삽입하는 과정

검색하는 방법과 마찬가지로 삽입할 위치를 찾습니다. 추가할 값인 5는 4보다 크고, 4의 오른쪽 자식 노드는 비어 있으므로 해당 위치를 삽입 위치로 선택합니다.



4의 오른쪽 자식 노드로 5를 삽입합니다.



[그림 9-12] 이진검색트리에 노드를 삽입하는 과정



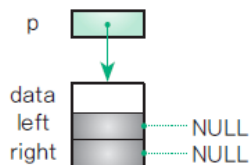
09-2 이진트리와 이진검색트리

이진검색트리 만들기 - (7)

■ 노드를 삽입하는 Add 함수 - (2)

- p를 루트로 하는 서브 트리에 대해 x가 가리키는 Member형 객체를 삽입
- 1 p가 널인 경우
 - Add 함수는 재귀 함수이므로 처음에 Add 함수를 호출할 때 p가 NULL이면 루트 노드가 없다는 뜻
 - 따라서 노드를 만들고 값을 설정하면 루트만 있는 이진검색trie가 만들어지고 삽입 과정이 끝남
 - 만약 함수를 처음 호출하는 상태가 아니라면(재귀 호출 시에 p값이 널이면) 노드를 삽입할 위치를 제대로 찾은 것
 - 삽입할 노드를 만들고 값을 설정하면 삽입 과정이 끝남

1. node에 루트 포인터를 대입합니다(루트를 선택합니다).
2. 삽입할 키 key와 node의 키값을 비교합니다. 값이 같다면 삽입에 실패합니다(종료).
 - 값이 같지 않은 경우 key값이 삽입할 값보다 작으면
왼쪽 자식 노드가 없는 경우에는(가) 노드를 삽입합니다(종료).
 - 왼쪽 자식 노드가 있는 경우에는 선택한 노드에 왼쪽 자식 노드 포인터를 대입합니다.
 - key값이 삽입할 값보다 크면
오른쪽 자식 노드가 없는 경우에는(나) 노드를 삽입합니다(종료).
 - 오른쪽 자식 노드가 있는 경우에는 선택한 노드에 오른쪽 자식 노드 포인터를 대입합니다.
3. 2로 되돌아갑니다.



[그림 9-13] 루트만 있는 이진검색트리

Do it! 실습 9-2[C]

• 완성 파일 chap09/BinTree.c

```

01  /*--- 노드 삽입 ---*/
02  BinNode *Add (BinNode *p, const Member *x)
03  {
04      int cond;
05      if(p == NULL) {
06          p = AllocBinNode();
07          SetBinNode(p, x, NULL, NULL);
08      } else if((cond = MemberNocmp(x, &p->data)) == 0)
09          printf("【 오류 】 %d는 이미 등록되어 있습니다.\n", x->no);
10      else if(cond < 0)
11          p->left = Add(p->left, x);
12      else
13          p->right = Add(p->right, x);
14      return p;
15  }
16
  
```

(실습 9-2[D]에서 계속)



09-2 이진트리와 이진검색트리

이진검색트리 만들기 - (8)

노드를 삽입하는 Add 함수 - (2)

- 2 p가 널이 아닌 경우
 - 3 선택한 노드의 키값과 삽입할 키값이 같은 경우
 - 값을 삽입할 수 없으므로 삽입할 수 없다는 내용을 출력
 - 4 삽입할 키값이 선택한 노드의 키값보다 작은 경우
 - 선택한 노드에 왼쪽 자식 노드를 대입
 - 실제로는 Add 함수에 왼쪽 자식 노드를 전달하며 재귀 호출
 - 5 삽입할 키값이 선택한 노드의 키값보다 큰 경우
 - 선택한 노드에 오른쪽 자식 노드를 대입
 - 실제로는 Add 함수에 오른쪽 자식 노드를 전달하며 재귀 호출

Do it! 실습 9-2[C]

• 완성 파일 chap09/BinTree.c

```
01  /*--- 노드 삽입 ---*/
02  BinNode *Add (BinNode *p, const Member *x)
03  {
04      int cond;
05      if(p == NULL) {
06          p = AllocBinNode();
07          SetBinNode(p, x, NULL, NULL);
08      } else if((cond = MemberNocmp(x, &p->data)) == 0)
09          printf("【 오류 】 %d는 이미 등록되어 있습니다.\n", x->no);
10      else if(cond < 0)
11          p->left = Add(p->left, x);
12      else
13          p->right = Add(p->right, x);
14      return p;
15  }
16
```

(실습 9-2[D]에서 계속)



09-2 이진트리와 이진검색트리

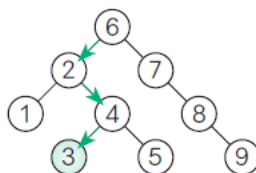
이진검색트리 만들기 - (9)

■ 노드를 삭제하는 Remove 함수 - (1)

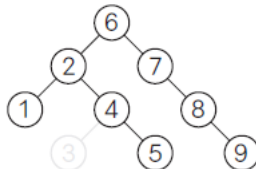
- 노드를 삭제할 때는 '세 가지 서로 다른 상황'에 놓이기 때문에 각각의 상황에 맞게 처리
- [A] 자식 노드가 없는 노드를 삭제하는 경우
 - 삭제할 노드가 부모 노드의 왼쪽 자식이면 부모의 왼쪽 포인터를 NULL로 함
 - 삭제할 노드가 부모 노드의 오른쪽 자식이면 부모의 오른쪽 포인터를 NULL로 함

a 3을 삭제하는 경우

검색과 마찬가지로 3을 먼저 찾아갑니다. 그런 다음 삭제할 노드 3에서 멈춥니다.

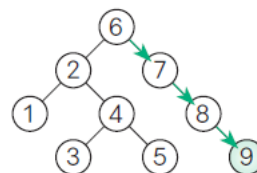


부모의 왼쪽 포인터에 NULL을 대입합니다.

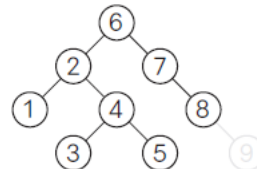


b 9를 삭제하는 경우

검색과 마찬가지로 9를 먼저 찾아갑니다. 그런 다음 삭제할 노드 9에서 멈춥니다.



부모의 오른쪽 포인터에 NULL을 대입합니다.



[그림 9-14] 자식 노드가 없는 노드를 삭제하는 과정



09-2 이진트리와 이진검색트리

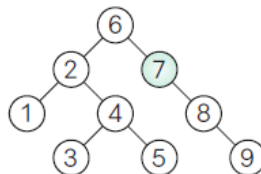
이진검색트리 만들기 - (10)

■ [B] 자식 노드가 1개인 노드를 삭제하는 경우

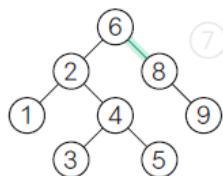
- 삭제 대상 노드가 부모 노드의 왼쪽 자식인 경우 부모의 왼쪽 포인터가 삭제 대상 노드의 자식을 가리키도록 함
- 삭제 대상 노드가 부모 노드의 오른쪽 자식인 경우 부모의 오른쪽 포인터가 삭제 대상 노드의 자식을 가리키도록 함

a 7을 삭제하는 경우

검색과 마찬가지로 7을 찾아갑니다. 그런 다음 삭제할 노드 7에서 멈춥니다.

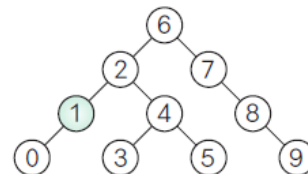


부모 노드인 6의 오른쪽 포인터가 7의 자식 노드인 8을 가리키도록 업데이트합니다.

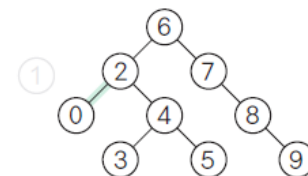


b 1을 삭제하는 경우

검색과 마찬가지로 1을 찾아갑니다. 그런 다음 삭제할 노드 1에서 멈춥니다.



부모 노드인 2의 왼쪽 포인터가 1의 자식 노드인 0을 가리키도록 업데이트합니다.



[그림 9-15] 자식 노드가 1개인 노드를 삭제하는 과정



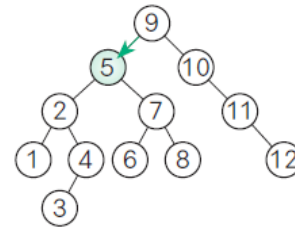
09-2 이진트리와 이진검색트리

이진검색트리 만들기 - (11)

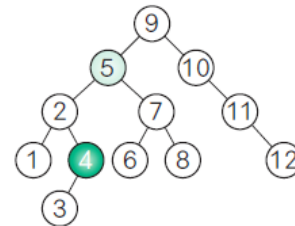
■ [C] 자식 노드가 2개인 노드를 삭제하는 경우

- 1. 삭제할 노드의 왼쪽 서브 트리에서 키값이 가장 큰 노드를 검색
- 2. 검색한 노드를 삭제 위치로 옮김(검색한 노드의 데이터를 삭제 대상 노드 위치로 복사)
- 3. 옮긴 노드를 삭제
 - 옮긴 노드에 자식이 없으면 [A] 자식 노드가 없는 노드를 삭제하는 경우'에 따라 노드를 삭제
 - 옮긴 노드에 자식이 1개만 있으면 [B] 자식 노드가 1개인 노드를 삭제하는 경우'에 따라 노드를 삭제

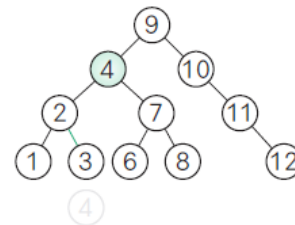
a 5를 삭제하는 경우



검색과 마찬가지로 5를 찾아갑니다. 그런 다음 삭제할 노드 5에서 멈춥니다.



노드 5의 왼쪽 서브 트리(노드 2가 루트)에서 가장 큰 키값을 갖는 노드를 검색합니다. 현재는 4가 가장 큰 값을 가지는 노드이므로 여기에서 멈춥니다.



5가 있는 곳으로 4를 옮기면 삭제 과정이 끝납니다. 4를 옮기려면 먼저 4의 데이터를 5의 위치로 복사합니다. 그런 다음 원래의 4를 트리에서 떼어내면 됩니다.

[그림 9-16] 자식 노드가 2개인 노드를 삭제하는 과정



09-2 이진트리와 이진검색트리

이진검색트리 만들기 - (12)

■ 노드를 삭제하는 Remove 함수 - (2)

- Search 함수와 Add 함수의 첫 번째 매개변수의 자료형은 BinNode *이지만 Remove 함수의 첫 번째 매개변수의 자료형은 BinNode **
- 이는 루트만 있는 이진검색트리에서 루트 노드를 삭제하는 경우에는 루트 포인터를 널로 업데이트하기 때문

Do it! 실습 9-2[D]

• 완성 파일 chap09/BinTree.c

```
01  /*--- 노드 삭제 ---*/
02  int Remove (BinNode **root, const Member *x)
03  {
04      BinNode *next *temp;
05      BinNode **left;

06      BinNode **p = root;
07
```

```
08      while(1) {
09          int cond;
10          if(*p == NULL) {
11              printf("【 오류 】 %d는 등록되어 있지 않습니다.\n", x->no);
12              return -1;          // 이 키는 없음
13          } else if((cond = MemberNocmp(x, &(*p)->data)) == 0)
14              break;              // 검색 성공
15          else if(cond < 0)
16              p = &((*p)->left);  // 왼쪽 서브 트리에서 검색
17          else
18              p = &((*p)->right); // 오른쪽 서브 트리에서 검색
19      }

21      if((*p)->left == NULL)
22          next = (*p)->right;
23      else {
24          left = &((*p)->left);
25          while((*left)->right != NULL)
26              left = &(*left)->right;
27          next = *left;
28          *left = (*left)->left;
29          next->left = (*p)->left;
30          next->right = (*p)->right;
31      }
32      temp = *p;
33      *p = next;
34      free(temp);
35
36      return 0;
37  }
38
```

(실습 9-2[E]에서 계속)



09-2 이진트리와 이진검색트리

이진검색트리 만들기 - (13)

■ 모든 노드를 출력하는 PrintTree 함수

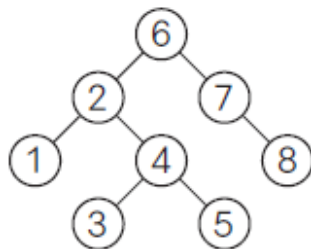
- 모든 노드를 키값의 오름차순으로 출력하는 함수
- 오름차순으로 출력하기 위해 중위 순회(inorder) 방법으로 트리를 검색
- 처음 호출할 때 매개변수 p에 루트 노드의 포인터를 전달받음
- 회원의 데이터 출력은 Member.c에서 정의한 PrintLnMember 함수로 처리
- 이진검색트리의 모든 데이터를 키값의 오름차순으로 출력
 - 1. 노드 2를 가리키는 왼쪽 포인터를 PrintTree 함수에 전달하면서 재귀호출
 - 2. 현재 노드의 데이터인 6을 출력
 - 3. 노드 7을 가리키는 오른쪽 포인터를 PrintTree 함수에 전달하면서 재귀호출

Do it! 실습 9-2[E]

• 완성 파일 chap10/BinTree.c

```
01  /*--- 모든 노드의 데이터를 출력 ---*/
02  void PrintTree (const BinNode *p)
03  {
04      if(p != NULL) {
05          PrintTree(p->left);
06          PrintLnMember(&p->data);
07          PrintTree(p->right);
08      }
09  }
10
```

(실습 9-2[F]에서 계속)



[그림 9-17] 이진검색트리



09-2 이진트리와 이진검색트리

이진검색트리 만들기 - (14)

- 모든 노드를 삭제하는 FreeTree 함수
 - 모든 노드를 삭제하는 함수
 - 후위 순회(postorder) 방법으로 트리를 검색하며 삭제를 진행

Do it! 실습 9-2[F]

• 완성 파일 chap09/BinTree.c

```
01  /*--- 모든 노드를 삭제 ---*/
02  void FreeTree (BinNode *p)
03  {
04      if(p != NULL) {
05          FreeTree(p->left);
06          FreeTree(p->right);
07          free(p);
08      }
09  }
```



09-2 이진트리와 이진검색트리

이진검색트리 만들기 - (15)

■ 실습 9-3 - (1)

- 이진검색트리를 사용하는 프로그램
- 이 프로그램을 컴파일할 때는 Member.h, Member.c, BinTree.h, BinTree.c가 필요

Do it! 실습 9-3

• 완성 파일 chap09/BinTreeTest.c

```
01 // 이진검색트리를 사용하는 프로그램
02 #include <stdio.h>
03 #include "Member.h" — 실습 10-1에서 작성
04 #include "BinTree.h"
05
06 /*--- 메뉴 ---*/
07 typedef enum {
08     TERMINATE, ADD, REMOVE, SEARCH, PRINT
09 } Menu;
10
11 /*--- 메뉴 선택 ---*/
12 Menu SelectMenu(void)
13 {
14     int ch;
15     do {
16         printf("(1)삽입 (2)삭제 (3)검색 (4)출력 (0)종료: ");
17
18         scanf("%d", &ch);
19     } while(ch < TERMINATE || ch > PRINT);
20     return(Menu)ch;
21 }
```



09-2 이진트리와 이진검색트리

이진검색트리 만들기 - (16)

■ 실습 9-3 - (2)

- 노드의 삽입, 삭제, 검색, 출력을 수행

```
22  /*--- 메인 함수 ---*/
23  int main (void)
24  {
25      Menu menu;
26      BinNode *root = NULL;      // 이진검색트리의 루트 노드 포인터
27      do {
28          Member x;
29          BinNode *temp;
30          switch(menu = SelectMenu()) {
31              /*--- 노드를 삽입 ---*/
32              case ADD :
33                  x = ScanMember("삽입", MEMBER_NO | MEMBER_NAME);
34                  root = Add(root, &x);
35                  break;
```

```
37          /*--- 노드를 삭제 ---*/
38          case REMOVE :
39              x = ScanMember("삭제", MEMBER_NO);
40              Remove(&root, &x);
41              break;
42
43          /*--- 노드를 검색 ---*/
44          case SEARCH :
45              x = ScanMember("검색", MEMBER_NO);
46              if((temp = Search(root, &x)) != NULL)
47                  PrintLnMember(&temp->data);
48              break;
49          /*--- 모든 노드를 출력 ---*/
50          case PRINT :
51              puts("【 모든 노드 출력 】");
52              PrintTree(root);
53              break;
54          }
55      } while(menu != TERMINATE);
56      FreeTree(root);
57      return 0;
58  }
```



09-2 이진트리와 이진검색트리

이진검색트리 만들기 - (17)

■ 실습 9-3 실행 결과

실행 결과

(1) 삽입 (2) 삭제 (3) 검색 (4) 출력 (0) 종료 : 1

삽입하는 데이터를 입력하세요.

번호 : 1 {1, 현규}를 삽입

이름 : 현규

(1) 삽입 (2) 삭제 (3) 검색 (4) 출력 (0) 종료 : 1

삽입하는 데이터를 입력하세요.

번호 : 10 {10, 진아}를 삽입

이름 : 진아

(1) 삽입 (2) 삭제 (3) 검색 (4) 출력 (0) 종료 : 1

삽입하는 데이터를 입력하세요.

번호 : 5 {5, 영준}을 삽입

이름 : 영준

(1) 삽입 (2) 삭제 (3) 검색 (4) 출력 (0) 종료 : 1

삽입하는 데이터를 입력하세요.

번호 : 12 {12, 윤미}를 삽입

이름 : 윤미

(1) 삽입 (2) 삭제 (3) 검색 (4) 출력 (0) 종료 : 1

삽입하는 데이터를 입력하세요.

번호 : 14 {14, 연의}를 삽입

이름 : 연의

(1) 삽입 (2) 삭제 (3) 검색 (4) 출력 (0) 종료 : 3

검색하는 데이터를 입력하세요.

번호 : 5 {5}를 검색

5 영준

(1) 삽입 (2) 삭제 (3) 검색 (4) 출력 (0) 종료 : 4

【 모든 노드 출력 】

1 현규

5 영준

10 진아

12 윤미

14 연의

키값의 오름차순으로 모든 노드 출력

(1) 삽입 (2) 삭제 (3) 검색 (4) 출력 (0) 종료 : 2

삭제하는 데이터를 입력하세요.

번호 : 10 {10}을 삭제

(1) 삽입 (2) 삭제 (3) 검색 (4) 출력 (0) 종료 : 4

【 모든 노드 출력 】

1 현규

5 영준

12 윤미

14 연의

키값의 오름차순으로 모든 노드 출력

(1) 삽입 (2) 삭제 (3) 검색 (4) 출력 (0) 종료 : 0

해시법

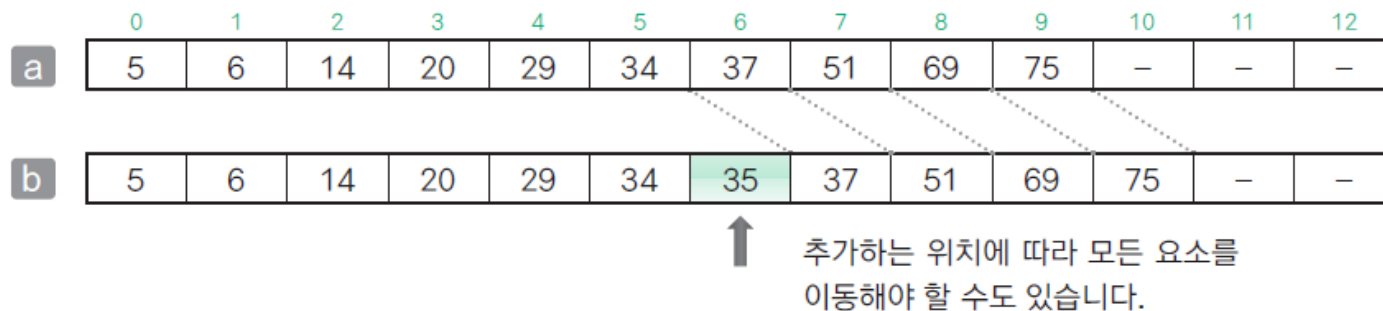


10-1 해시법

정렬된 배열에 새로운 값 추가하기

■ 정렬된 배열에 데이터를 추가

- a 배열의 앞쪽 10개의 요소에 데이터가 오름차순으로 정렬된 상태로 저장
- 배열에 35를 추가하려면 아래와 같은 작업이 필요
 - 1. 삽입할 위치가 a[5]와 a[6] 사이임을 이진검색법으로 조사
 - 2. 그림 b와 같이 a[6] 이후의 모든 요소를 하나씩 뒤로 이동
 - 3. a[6]에 35를 대입
- 요소 이동에 필요한 복잡도(time-complexity)는 $O(n)$ 이며 이 비용은 적은 비용이 아님



[그림 10-1] 정렬된 배열에 데이터를 추가



10-1 해시법

해시법 정의하기 – (1)

■ 해시법 hashing – (1)

- 데이터를 저장할 위치(인덱스)를 간단한 연산으로 구하는 것으로, 검색뿐만 아니라 추가, 삭제도 효율적으로 수행하는 방법
- 그림 10-2의 a에서 볼 수 있는 배열의 키값(각 요솟값)을 배열의 요소 개수 13으로 나눈 나머지로 다시 정리하면 표 10-1과 같음
- 표에 정리한 값을 해시값(hash value)이라고 하며, 이 해시값은 데이터에 접근할 때 사용
- 해시값이 인덱스가 되도록 원래의 키값을 저장한 배열이 해시 테이블(hash table)

[표 10-1] 키값과 해시값의 대응

키값	5	6	14	20	29	34	37	51	69	75
해시값(13으로 나눈 나머지)	5	6	1	7	3	8	11	12	4	10

	0	1	2	3	4	5	6	7	8	9	10	11	12
a	-	14	-	29	69	5	6	20	34	-	75	37	51
b	-	14	-	29	69	5	6	20	34	35	75	37	51

새로운 값(35)을 추가하더라도 다른 요소가 이동할
필요가 없습니다. ↑

[그림 10-2] 해시에 새로운 값(35)을 추가



10-1 해시법

해시법 정의하기 - (2)

■ 해시법 hashing - (2)

- 배열에 35를 추가하는 경우 35를 13으로 나눈 나머지는 9이므로 **b**처럼 a[9]에 값(35)을 저장
- 이전의 '추가한 값' 이후의 배열 요소를 모두 옮겼던 경우와는 다르게 새로운 값을 추가하더라도 다른 배열 요소를 뒤로 옮기지 않아도 됨
- 이렇게 키값(35)을 가지고 해시값(9)을 만드는 과정을 해시 함수(hash function)라고 함
- 해시 함수는 여기에서 살펴봤듯이 '나머지를 구하는 연산 또는 이런 나머지 연산을 다시 응용한 연산'을 사용
- 해시 테이블의 각 요소를 버킷(bucket)이라고 함

[표 10-1] 키값과 해시값의 대응

키값	5	6	14	20	29	34	37	51	69	75
해시값(13으로 나눈 나머지)	5	6	1	7	3	8	11	12	4	10

	0	1	2	3	4	5	6	7	8	9	10	11	12
a	-	14	-	29	69	5	6	20	34	-	75	37	51
b	-	14	-	29	69	5	6	20	34	35	75	37	51

새로운 값(35)을 추가하더라도 다른 요소가 이동할
필요가 없습니다. ↑

[그림 10-2] 해시에 새로운 값(35)을 추가



10-1 해시법

충돌 살펴보기 - (1)

■ 충돌collision

- 저장할 버킷이 중복되는 현상
- 배열에 새로운 값 18을 추가하는 경우 18을 13으로 나눈 나머지인 해시값은 5이고 저장할 곳은 버킷 a[5]
- 그런데 그림 10-3처럼 이 버킷은 이미 채워져 있음
- 키값과 해시값의 대응 관계가 반드시 1대1이라는 보장은 없음(보통 n대1)

■ 충돌에 대한 대처

- 1. 체인법: 같은 해시값을 갖는 요소를 연결 리스트로 관리
- 2. 오픈 주소법: 빈 버킷을 찾을 때까지 해시를 반복

0	1	2	3	4	5	6	7	8	9	10	11	12
-	14	-	29	69	5	6	20	34	-	75	37	51

↑ 삽입할 a[5]는 이미 채워져 있습니다.

18

[그림 10-3] 해시에 추가할 때의 충돌



10-1 해시법

Do it! 실습 10-1

• 완성 파일 chap10/Member.h

충돌 살펴보기 - (2)

■ 키값과 데이터

- 해시법을 사용하는 프로그램에서 다루는 데이터는 단순한 정수나 실수가 아니라 여러 데이터가 결합된 '구조체'인 경우가 적지 않음
- 체인법과 오픈 주소법을 배우기 전에 프로그램에서 사용할 데이터인 구조체를 먼저 정의

■ 실습 10-1

- 스포츠클럽의 회원에 대한 정보
 - 번호는 int형 정수, 이름은 문자열로 하고 이 두 가지의 자료형을 결합한 구조체를 Member라고 함
 - 회원 정보를 출력하는 함수와 멤버의 값을 입력하는 함수가 필요하므로 두 함수를 미리 만들어둠

```
01 // 회원 데이터 Member(헤더)
02 #ifndef __Member
03 #define __Member
04
05 /*--- 회원 데이터 ---*/
06 typedef struct {
07     int no;                // 번호
08     char name[20];         // 이름
09 } Member;
10
11 #define MEMBER_NO 1        // 번호를 나타내는 정숫값
12 #define MEMBER_NAME 2     // 이름을 나타내는 정숫값
13
14 /*--- 회원 번호 비교 함수 ---*/
15 int MemberNoCmp(const Member *x, const Member *y);
16
17 /*--- 회원 이름 비교 함수 ---*/
18 int MemberNameCmp(const Member *x, const Member *y);
19
20 /*--- 회원 데이터를 출력(줄 바꿈 없음) ---*/
21 void PrintMember(const Member *x);
22
23 /*--- 회원 데이터를 출력(줄 바꿈 있음) ---*/
24 void PrintLnMember(const Member *x);
25
26 /*--- 회원 데이터를 읽음 ---*/
27 Member ScanMember(const char *message, int sw);
28 #endif
```



10-1 해시법

충돌 살펴보기 - (3)

■ 실습 10-2 - (1)

- MemberNoCmp 함수: 2개의 회원 데이터에서 '번호'의 대소 관계를 판단하는 비교 함수
- MemberNameCmp 함수: 2개의 회원 데이터에서 '이름'의 대소 관계를 판단하는 비교 함수

Do it! 실습 10-2

• 완성 파일 chap10/Member.c

```
01 // 회원 데이터 Member(소스)
02 #include <stdio.h>
03 #include <string.h>
04 #include "Member.h"
05
06 /*--- 회원 번호 비교 함수 ---*/
07 int MemberNoCmp(const Member *x, const Member *y)
08 {
09     return x->no < y->no ? -1: x->no > y->no ? 1: 0;
10 }
11
12 /*--- 회원 이름 비교 함수 ---*/
13 int MemberNameCmp(const Member *x, const Member *y)
14 {
15     return strcmp(x->name, y->name);
16 }
```



10-1 해시법

충돌 살펴보기 - (4)

■ 실습 10-2 - (2)

- PrintMember 함수: 번호와 이름을 출력하는 함수
- PrintLnMember 함수: 번호와 이름을 출력하는 함수로 끝에 줄 바꿈 문자를 출력
- ScanMember 함수: 번호와 이름 가운데 하나 혹은 둘 모두를 대화형으로 읽는 함수

```
18  /*--- 회원 데이터(번호와 이름)를 출력(줄 바꿈 없음) ---*/
19  void PrintMember(const Member *x)
20  {
21      printf("%d %s", x->no, x->name);
22  }
23
24  /*--- 회원 데이터(번호와 이름)를 출력(줄 바꿈 있음) ---*/
25  void PrintLnMember(const Member *x)
26  {
27      printf("%d %s\n", x->no, x->name);
28  }
29
30  /*--- 회원 데이터(번호와 이름)를 읽음 ---*/
31  Member ScanMember(const char *message, int sw)
32  {
33      Member temp;
34      printf("%s하는 데이터를 입력하세요.\n", message);
35      if(sw & MEMBER_NO) {printf("번호: "); scanf("%d", &temp.no);}
36      if(sw & MEMBER_NAME) {printf("이름: "); scanf("%s", temp.name);}
37      return temp;
38  }
```




10-1 해시법

체인법 살펴보기 - (1)

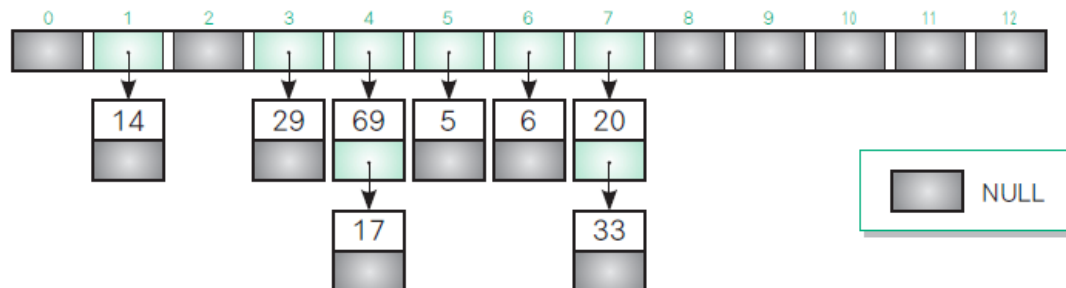
■ 체인법 chaining

- 해시값을 갖는 데이터를 쇠사슬(chain) 모양으로 연결 리스트에서 연결하는 방법으로, 오픈 해시법(open hashing)이라고도 함

■ 같은 해시값을 갖는 데이터 저장하기

- 배열의 각 버킷(해시 테이블)에 저장하는 값은 그 인덱스를 해시값으로 하는 연결 리스트의 첫 번째 노드(node)에 대한 포인터
 - 그림 10-4에서 69와 17의 해시값은 모두 4이며, 이들을 연결하는 연결 리스트의 첫 번째 노드에 대한 포인터를 table[4]에 저장
 - 또 해시값(인덱스) 0과 2처럼 데이터가 하나도 없는 버킷의 값은 널(NULL) 포인터값을 저장

각 버킷에 저장하는 값은 '같은 해시값을 갖는 노드를 연결한 리스트'의 첫 번째 노드에 대한 포인터입니다.



같은 해시값을 갖는 데이터를 연결 리스트(사슬 모양)로 연결합니다.



10-1 해시법

체인법 살펴보기 - (2)

■ 버킷용 구조체 Node

- 개별 버킷의 자료형을 나타낸 것
 - data... 데이터
 - next... 다음 노드에 대한 포인터로 다음 노드가 존재하지 않을 때는 NULL이 됨

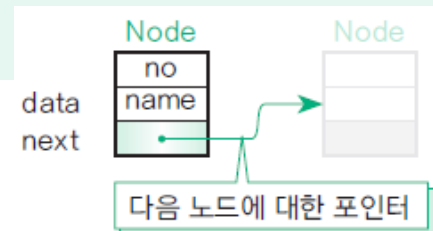
■ 해시 테이블을 관리하는 구조체 ChainHash

- size... 해시 테이블의 크기
- table... 해시 테이블을 저장하는 배열의 첫 번째 요소에 대한 포인터
- 멤버 table은 해시 테이블을 저장하는 배열(즉, 배열의 첫 번째 요소에 대한 포인터)
- 배열용 메모리 영역을 확보하는 작업은 Initialize 함수가 수행

Do it! 실습 10-3

• 완성 파일 chap10/ChainHash.h

```
01 // 체인법으로 구현한 해시(헤더)
02 #ifndef __ChainHash
03 #define __ChainHash
04
05 #include "Member.h"
06
07 /*--- 버킷을 나타내는 노드 ---*/
08 typedef struct __node {
09     Member      data;    // 데이터
10     struct __node *next; // 다음 노드에 대한 포인터
11 } Node;
12
13 /*--- 해시 테이블 ---*/
14 typedef struct {
15     int      size;        // 해시 테이블의 크기
16     Node **table;         // 해시 테이블의 첫 번째 요소에 대한 포인터
17 } ChainHash;
18
19 /*--- 해시 테이블 초기화 ---*/
20 int Initialize(ChainHash *h, int size);
```



[그림 10-5] 버킷을 나타내는 구조체



10-1 해시법

체인법 살펴보기 - (3)

■ 실습 10-3

- 체인법으로 구현한 해시(헤더)

```
22  /*--- 검색 ---*/
23  Node *Search(const ChainHash *h, const Member *x);
24
25  /*--- 데이터 추가 ---*/
26  int Add(ChainHash *h, const Member *x);
27
28  /*--- 데이터 삭제 ---*/
29  int Remove(ChainHash *h, const Member *x);
30
31  /*--- 해시 테이블 덤프(dump) ---*/
32  void Dump(const ChainHash *h);
33
34  /*--- 모든 데이터 삭제 ---*/
35
36  void Clear(ChainHash *h);
37
38  /*--- 해시 테이블 종료 ---*/
39  void Terminate(ChainHash *h);
40  #endif
```



10-1 해시법

체인법 살펴보기 - (4)

■ 해시값을 구하는 hash 함수

- 매개변수 key로 받은 회원 번호 값을 해시 테이블의 크기 size로 나눈 나머지를 반환

■ 노드에 값을 설정하는 SetNode 함수

- 두 번째 인수 x가 가리키는 데이터와, 세 번째 인수 n로 전달받은 다음 노드에 대한 포인터 next를, 첫 번째 인수 n이 가리키는 노드의 각 멤버에 저장
- 구조체에서 매개변수를 주고받을 때는 값이 아니라 포인터로 하는 것이 정석인데
- 함수 사이에 주고받는 데이터의 크기(바이트 수)에 제한이 있기 때문

Do it! 실습 10-4[A]

• 완성 파일 chap10/ChainHash.c

```
01 // 체인법으로 구현한 해시(소스)
02 #include <stdio.h>
03 #include <stdlib.h>
04 #include "Member.h"
05 #include "ChainHash.h"
06
07 /*--- 해시 함수(key의 해시값을 반환) ---*/
08 static int hash(int key, int size)
09 {
10     return key % size;
11 }
12
13 /*--- 노드의 각 멤버에 값을 설정 ---*/
14 static void SetNode(Node *n, const Member *x, const Node *next)
15 {
16     n->data = *x;    // 데이터
17
18     n->next = next;    // 다음 노드에 대한 포인터
19 }
```



10-1 해시법

체인법 살펴보기 - (5)

■ 해시 테이블을 초기화하는 Initialize 함수

- 첫 번째 인수 h는 처리 대상인 해시 구조체 객체에 대한 포인터
- 요소의 개수가 size인 배열 table의 본체를 생성하고 매개변수 size로 받은 값을 멤버 size에 복사
- 배열 table의 모든 요소에 공백 포인터 NULL을 대입했기 때문에 그림 10-6처럼 모든 버킷이 공백 상태
- 23~26행에서는 메모리 영역 확보에 실패 (calloc 함수가 NULL을 반환)하면 멤버 table에 NULL을 대입하고, 멤버 size에 0을 대입

```
20  /*--- 해시 테이블 초기화 ---*/
21  int Initialize(ChainHash *h, int size)
22  {
23      if((h->table = calloc(size, sizeof(Node *))) == NULL) {
24          h->size = 0;
25          return 0;
26      }
27      h->size = size;
28      for(int i = 0; i < size; i++) // 모든 버킷을 공백(NULL) 상태로 변경
29          h->table[i] = NULL;
30      return 1;
31  }
32
```

(실습 10-4[B]에서 계속)



모든 버킷이 널(NULL)입니다.

[그림 10-6] 모든 버킷이 널인 해시



10-1 해시법

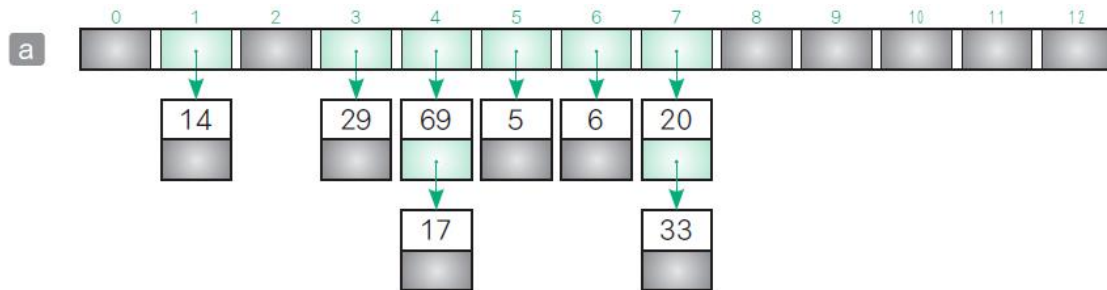
체인법 살펴보기 - (6)

- 키값으로 요소를 검색하는 Search 함수
 - 키값이 x->no인 요소를 검색하는 함수
 - 1. 해시 함수가 키값을 해시값으로 변환
 - 2. 해시값을 인덱스로 하는 버킷을 선택
 - 3. 선택한 버킷의 연결 리스트를 처음부터 순서대로 검색
 - 키값과 같은 값을 찾으면 검색 성공
 - 끝까지 스캔하여 찾지 못하면 검색 실패

Do it! 실습 10-4[B]

• 완성 파일 chap10/ChainHash.c

```
01  /*--- 검색 ---*/
02  Node *Search(const ChainHash *h, const Member *x)
03  {
04      int key = hash(x->no, h->size); // 검색하는 데이터의 해시값
05      Node *p = h->table[key];        // 현재 선택한 노드
06
07      while(p != NULL) {
08          if(p->data.no == x->no)      // 검색 성공
09              return p;
10          p = p->next;                // 다음 노드를 선택
11      }
12      return NULL;                    // 검색 실패
13  }
```





10-1 해시법

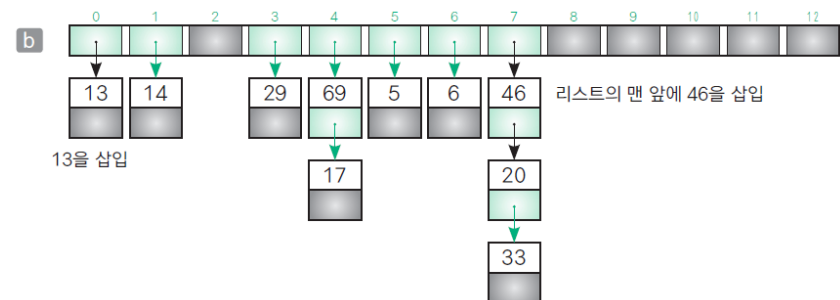
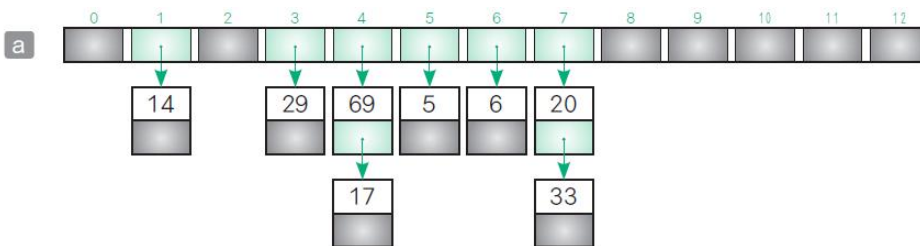
체인법 살펴보기 - (7)

■ 요소를 추가하는 Add 함수

- 포인터 x가 가리키는 데이터를 추가하는 함수
- 1. 해시 함수가 키값을 해시값으로 변환
- 2. 해시값을 인덱스로 하는 버킷을 선택
- 3. 버킷에 저장된 포인터가 가리키는 연결 리스트를 처음부터 순서대로 검색
 - 키값과 같은 값을 찾으면 키값이 이미 등록된 상태이므로 추가에 실패
 - 끝까지 스캔하여 찾지 못하면 리스트의 맨 앞 위치에 노드를 삽입

```
15  /*--- 데이터 추가 ---*/
16  int Add(ChainHash *h, const Member *x)
17  {
18      int key = hash(x->no, h->size); // 추가하는 데이터의 해시값
19      Node *p = h->table[key];       // 현재 선택한 노드
20      Node *temp;
21      while(p != NULL) {
22          if(p->data.no == x->no)      // 이 키는 이미 등록됨
23              return 1;               // 추가 실패
24          p = p->next;                 // 다음 노드를 선택
25      }
26      if((temp = calloc(1, sizeof(Node))) == NULL)
27          return 2;
28      SetNode(temp, x, h->table[key]); // 추가하는 노드에 값을 설정
29      h->table[key] = temp;
30      return 0;                       // 추가 성공
31  }
32
```

(실습 10-4[C]에서 계속)



[그림 10-7] 체인법에 의한 해시의 검색과 추가



10-1 해시법

체인법 살펴보기 - (8)

■ 요소를 삭제하는 Remove 함수

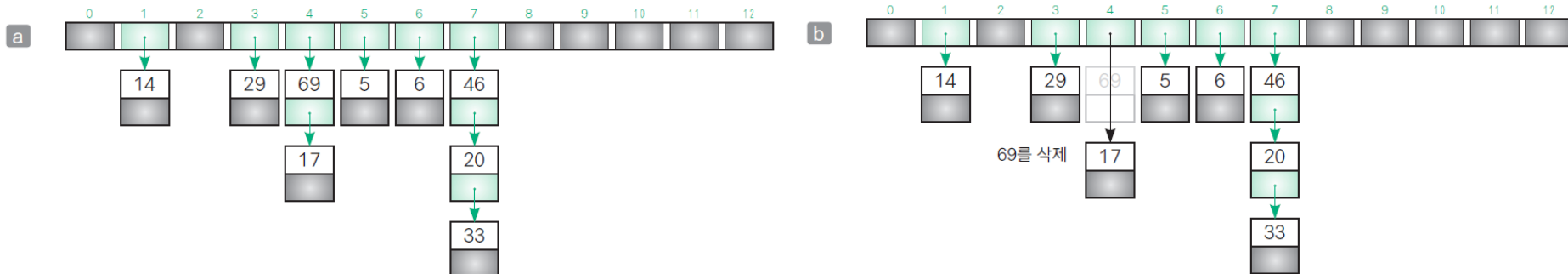
- 키값이 $x \rightarrow no$ 인 요소를 삭제하는 함수
- 1. 해시 함수가 키값을 해시값으로 변환
- 2. 해시값을 인덱스로 하는 버킷을 선택
- 3. 선택한 버킷의 포인터가 가리키는 연결 리스트를 처음부터 순서대로 검색
 - 키값과 같은 값을 찾으면 그 노드를 리스트에서 삭제
 - 그렇지 않으면 삭제에 실패

Do it! 실습 10-4[C]

• 완성 파일 chap10/ChainHash.c

```
01  /*--- 데이터 삭제 ---*/
02  int Remove(ChainHash *h, const Member *x)
03  {
04      int key = hash(x->no, h->size);    // 삭제하는 데이터의 해시값
05      Node *p = h->table[key];           // 현재 선택한 노드
06      Node **pp = &h->table[key];        // 현재 선택한 노드에 대한 포인터
07
08      while (p != NULL) {
09          if (p->data.no == x->no) {      // 찾으면
10              *pp = p->next;
11              free(p);                   // 해제
12              return 0;                  // 삭제 성공
13          }
14          pp = &p->next;
15          p = p->next;                   // 다음 노드를 선택
16      }
17
18      return 1;                           // 삭제 실패(존재하지 않음)
19  }
20
```

(실습 10-4[D]에서 계속)



[그림 10-8] 체인법에 의한 해시에서의 삭제



10-1 해시법

체인법 살펴보기 - (9)

■ 해시 테이블의 내용을 출력하는 Dump 함수

- 해시 테이블의 모든 요소(table[0]~table[size-1])에 대하여 다음에 오는 노드를 끌어당기면서 각 노드의 키값과 데이터를 출력하는 작업을 반복
- 같은 해시값을 갖는 데이터는 화살표(→)로 연결
- 같은 해시값을 갖는 버킷이 화살표로 연결된 것을 확인할 수 있음

Do it! 실습 10-4[D]

• 완성 파일 chap10/ChainHash.c

```
01  /*--- 해시 테이블 덤프 ---*/
02  void Dump(const ChainHash *h)
03  {
04      for (int i = 0; i < h->size; i++) {
05          Node *p = h->table[i];
06          printf("%02d ", i);
07          while (p != NULL) {
08              printf("→ %d (%s) ", p->data.no, p->data.name);
09              p = p->next;
10          }
11          putchar('\n');
12      }
13  }
14
```

```
00
01  → 14
02
03  → 29
04  → 69  → 17
05  → 5
06  → 6
07  → 46  → 20  → 33
08
09
10
11
12
```



10-1 해시법

체인법 살펴보기 - (10)

■ 모든 데이터를 삭제하는 Clear 함수

- 1. 배열 표의 요소가 NULL이 아니면 그 해시값을 갖는 데이터가 연결 리스트로 존재하므로, 연결 리스트를 맨 앞부터 순서대로 검사하면서 모든 노드에 대한 메모리를 해제
- 2. 검사 중인 배열 요소에 NULL을 대입
- 3. 배열에 대한 모든 검사가 끝나면 모든 버킷이 '공백' 상태가 됨

■ 해시 테이블을 종료하는 Terminate 함수

- 1. Clear 함수로 해시에 등록한 모든 데이터를 삭제
- 2. Initialize 함수로 메모리에 확보한 해시 테이블(table)을 해제
- 3. 해시 테이블의 크기를 저장하는 멤버 size를 0으로 초기화

Do it! 실습 10-4대

• 완성 파일 chap10/ChainHash.c

```
01  /*--- 모든 데이터 삭제 ---*/
02  void Clear(ChainHash *h)
03  {
04      for (int i = 0; i < h->size; i++) {
05          Node *p = h->table[i];      // 현재 선택한 노드
06          while (p != NULL) {
07              Node *next = p->next;
08              free(p);                // 현재 선택한 노드의 메모리 해제 1
09              p = next;              // 다음 노드 선택
10          }
11          h->table[i] = NULL;        2
12      }
13  }
14
15  /*--- 해시 테이블 종료 ---*/
16  void Terminate(ChainHash *h)
17  {
18      Clear(h);                      // 모든 데이터 삭제
19      free(h->table);                // 해시 테이블 배열의 메모리 해제
20      h->size = 0;                    // 해시 테이블 크기를 0으로 초기화(clear)
21  }
```



10-1 해시법

체인법 살펴보기 - (11)

- 실습 10-5 - (1)
 - 체인법을 사용하는 프로그램

Do it! 실습 10-5

• 완성 파일 chap10/ChainHashTest.c

```
01 // 체인 해시(ChainHash)의 사용
02 #include <stdio.h>
03 #include "Member.h"
04 #include "ChainHash.h"
05
06 /*--- 메뉴 ---*/
07 typedef enum {
08     TERMINATE, ADD, DELETE, SEARCH, CLEAR, DUMP
09 } Menu;
10
11 /*--- 메뉴 선택 ---*/
12 Menu SelectMenu(void)
13 {
14     int ch;
15     do {
16
17         printf("(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: ");
18         scanf("%d", &ch);
19     } while(ch < TERMINATE || ch > DUMP);
20     return(Menu)ch;
21 }
```



10-1 해시법

체인법 살펴보기 - (12)

■ 실습 10-5 - (2)

```
22  /*--- 메인 ---*/
23  int main(void)
24  {
25      Menu menu;          // 메뉴
26      ChainHash hash;      // 해시 테이블
27      Initialize(&hash, 13); // 해시 테이블 초기화
28      do {
29          int result;
30          Member x;
31          Node *temp;
32          switch(menu = SelectMenu()) {
33              case ADD:          /*--- 데이터 추가 ---*/
34                  x = ScanMember("추가", MEMBER_NO | MEMBER_NAME);
35                  result = Add(&hash, &x);
36                  if(result)
37                      printf("\a오류: 추가에 실패했습니다(%s).\n",
38                          (result == 1) ? "이미 등록됨": "메모리 부족");
39                  break;
40
41              case DELETE:       /*--- 데이터 삭제 ---*/
42                  x = ScanMember("삭제", MEMBER_NO);
43                  result = Remove(&hash, &x);
44                  if(result == 1)
45                      printf("\a오류: 이 번호의 데이터가 존재하지 않습니다.\n");
46                  break;
```

```
48          case SEARCH:         /*--- 데이터 검색 ---*/
49              x = ScanMember("검색", MEMBER_NO);
50              temp = Search(&hash, &x);
51              if(temp == NULL)
52                  printf("\a검색에 실패했습니다.\n");
53              else {
54                  printf("검색에 성공했습니다.: ");
55                  PrintLnMember(&temp->data);
56              }
```

```
57          break;
58
59          case CLEAR:           /*--- 모든 데이터 삭제 ---*/
60              Clear(&hash);
61              break;
62
63          case DUMP:            /*--- 해시 테이블 덤프 ---*/
64              Dump(&hash);
65              break;
66          }
67      } while(menu != TERMINATE);
68      Terminate(&hash);        // 해시 테이블 종료
69
70      return 0;
71  }
```



10-1 해시법

체인법 살펴보기 - (13)

■ 실습 10-5 실행 결과 - (1)

실행 결과	
(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 1 추가할 데이터를 입력하세요. 번호: 1 이름: 붉은꼬리	{1 붉은꼬리}를 추가
(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 1 추가할 데이터를 입력하세요. 번호: 5 이름: 박현규	{5 박현규}를 추가
(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 1 추가할 데이터를 입력하세요. 번호: 10 이름: 지나	{10 지나}를 추가
(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 1 추가할 데이터를 입력하세요. 번호: 12 이름: 김영준	{12 김영준}를 추가

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 1
추가할 데이터를 입력하세요.

번호: 14 {14 최윤미}를 추가
이름: 최윤미

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 3
검색할 데이터를 입력하세요.
번호: 5 5를 검색
검색에 성공했습니다: 5 박현규

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 5

00 :
01 : → 14(최윤미) → 1(붉은꼬리) 같은 해시값을 갖는 데이터가 링크되어 있습니다.
02 :
03 :
04 :
05 : → 5(박현규)
06 : 해시 테이블의 내용을 출력
07 :
08 :
09 :
10 : → 10(지나)
11 :
12 : → 12(김영준)



10-1 해시법

체인법 살펴보기 – (14)

■ 실습 10-5 실행 결과 – (2)

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 2

삭제할 데이터를 입력하세요.

번호: 14

14를 삭제

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 5

00:
01: → 1(붉은꼬리)
02:
03:
04:
05: → 5(박현규)
06:
07:
08:
09:
10: → 10(지나)
11:
12: → 12(김영준)

해시 테이블의 내용을 출력

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 0



10-1 해시법

오픈 주소법 이해하기 - (1)

오픈 주소법 open addressing

- 충돌이 발생했을 때 재해시(rehashing)를 수행하여 비어 있는 버킷을 찾아내는 방법으로, 닫힌 해시법(closed hashing)이라고도 함
- 빈 버킷을 만날 때까지 재해시(rehashing)를 여러 번 반복하므로 연결 탐사법(linear probing)이라고도 함

요소 삽입

- a는 새로운 값(18)을 삽입하고자 할 때 충돌이 발생한 경우
- 이럴 때 사용하는 방법이 재해시(rehashing)
 - 재해시를 하면 $(18 + 1) \% 13$ 의 결과값(6)을 얻을 수 있음
- b처럼 인덱스가 6인 버킷도 데이터가 채워져 있으므로 재해시
 - $(19 + 1) \% 13$ 의 결과값(7)을 얻을 수 있음
- c처럼 인덱스가 7인 버킷에 새로운 데이터(18)를 삽입



[그림 10-9] 오픈 주소법을 이용한 재해시



10-1 해시법

오픈 주소법 이해하기 - (2)

■ 요소 삭제

- 인덱스가 5인 값을 삭제하려면 인덱스가 5인 버킷의 데이터를 비우면 될 것 같지만 실제로는 그렇지 않음
- 왜냐하면 같은 해시값을 갖는 18을 검색할 때 '해시값이 5인 데이터는 존재하지 않는다'라고 생각하여 검색에 실패하기 때문
- 그래서 각 버킷에 대해 아래의 속성을 부여
 - 1. 데이터 저장 속성값
 - 2. 비어 있음 속성값(-)
 - 3. 삭제 마침 속성값(★)
- 5를 삭제할 때 그림 10-10처럼 그 위치의 버킷에 삭제를 마쳤음을 나타내는 속성값으로 '★'을 저장

0	1	2	3	4	5	6	7	8	9	10	11	12
-	14	-	29	-	★	6	18	34	-	75	37	51

삭제 마침: 같은 해시값의 데이터가 다른 버킷에 저장되어 있습니다.

비어 있음: 같은 해시값의 데이터가 존재하지 않습니다.

[그림 10-10] 오픈 주소법의 삭제 마침 속성값과 비어 있음 속성값 사용

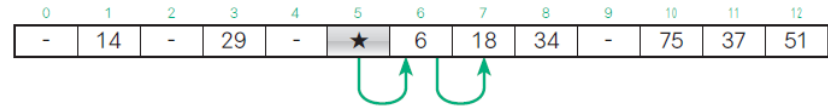


10-1 해시법

오픈 주소법 이해하기 - (3)

■ 요소 검색

- 값 17을 검색
 - 해시값이 4인 버킷을 보면 속성값이 '비어 있음(-)'이므로 검색 실패
- 값 18을 검색
 - 해시값이 5인 버킷을 보면 그 속성은 '삭제 마침(★)'
 - 그림 10-11처럼 재해시를 수행하여 6인 버킷을 다시 검색
 - 여기에는 값 6이 저장되어 있으므로 다시 재해시를 수행하여 7인 버킷을 검색
 - 검색하는 값 18이 저장되어 있으므로 검색 성공



원하는 값을 찾을 때까지 재해시를 반복합니다.

[그림 10-11] 오픈 주소법 검색



10-1 해시법

오픈 주소법 이해하기 - (4)

■ 실습 10-6

- 오픈 주소법으로 구현한 해시(헤더)

Do it! 실습 10-6

• 완성 파일 chap10/ClosedHash.h

```
01 // 오픈 주소법으로 구현한 해시(헤더)
```

```
02 #ifndef __ClosedHash
```

```
03 #define __ClosedHash
```

```
04
```

```
05 #include "Member.h"
```

```
06
```

```
07 /*--- 요소의 상태 ---*/
```

```
08 typedef enum {
```

```
09     Occupied, Empty, Deleted
```

```
10 } Status;
```

```
11
```

```
12 /*--- 요소 ---*/
```

```
13 typedef struct {
```

```
14     Member data;      // 데이터
```

```
15     Status stat;      // 요소의 상태
```

```
16 } Bucket;
```

```
17
```

```
18 /*--- 해시 테이블 ---*/
```

```
19 typedef struct {
```

```
20     int size;          // 해시 테이블의 크기
```

```
21     Bucket *table;     // 해시 테이블의 첫 번째 요소에 대한 포인터
```

```
22 } ClosedHash;
```

```
24 /*--- 해시 테이블 초기화 ---*/
```

```
25 int Initialize(ClosedHash *h, int size);
```

```
26
```

```
27 /*--- 검색 ---*/
```

```
28 Bucket *Search(const ClosedHash *h, const Member *x);
```

```
29
```

```
30 /*--- 데이터 추가 ---*/
```

```
31 int Add(ClosedHash *h, const Member *x);
```

```
32
```

```
33 /*--- 데이터 삭제 ---*/
```

```
34 int Remove(ClosedHash *h, const Member *x);
```

```
35
```

```
36 /*--- 해시 테이블 덤프 ---*/
```

```
37 void Dump(const ClosedHash *h);
```

```
38
```

```
39 /*--- 모든 데이터 삭제 ---*/
```

```
40 void Clear(ClosedHash *h);
```

```
42 /*--- 해시 테이블 종료 ---*/
```

```
43 void Terminate(ClosedHash *h);
```

```
44 #endif
```



10-1 해시법

오픈 주소법 이해하기 - (5)

■ 실습 10-7 - (1)

- 오픈 주소법으로 구현한 해시(소스)

Do it! 실습 10-7[A]

• 완성 파일 chap10/ClosedHash.c

```
01 // 오픈 주소법으로 구현한 해시(소스)
02
03 #include <stdio.h>
04 #include <stdlib.h>
05 #include "Member.h"
06 #include "ClosedHash.h"
07
08 /*--- 해시 함수(key의 해시값을 반환) ---*/
09 static int hash(int key, int size)
10 {
11     return key % size;
12 }
13 /*--- 재해시 함수 ---*/
14 static int rehash(int key, int size)
15 {
16     return (key + 1) % size;
17 }
```

```
34     for(int i = 0; i < size; i++)           // 모든 버킷을
35         h->table[i].stat = Empty;           // 공백으로 변경
36     return 1;
37 }
38
39 /*--- 노드의 각 멤버에 값을 설정 ---*/
40 static void SetBucket(Bucket *n, const Member *x, Status stat)
41 {
42     n->data = *x;           // 데이터
43     n->stat = stat;         // 요소의 상태
44 }
45
46 /*--- 해시 테이블 초기화 ---*/
47 int Initialize(ClosedHash *h, int size)
48 {
49     if((h->table = calloc(size, sizeof(Bucket))) == NULL) {
50         h->size = 0;
51         return 0;
52     }
53     h->size = size;
```



10-1 해시법

오픈 주소법 이해하기 - (6)

■ 실습 10-7 - (2)

```
39 /*--- 검색 ---*/
40 Bucket *Search(const ClosedHash *h, const Member *x)
41 {
42     int key = hash(x->no, h->size);    // 검색할 데이터의 해시값
43     Bucket *p = &h->table[key];        // 현재 선택한 노드
44     for(int i = 0; p->stat != Empty && i < h->size; i++) {
45         if(p->stat == Occupied && p->data.no == x->no)
46             return p;
47         key = rehash(key, h->size);    // 재해시
48         p = &h->table[key];
49     }
50     return NULL;
51 }
```

```
53 /*--- 데이터 추가 ---*/
54 int Add(ClosedHash *h, const Member *x)
55 {
56     int key = hash(x->no, h->size);    // 추가할 데이터의 해시값
57     Bucket *p = &h->table[key];        // 현재 선택한 노드
58     if(Search(h, x))                  // 이 키는 이미 등록됨
59         return 1;
60     for(int i = 0; i < h->size; i++) {
61         if(p->stat == Empty || p->stat == Deleted) {
62             SetBucket(p, x, Occupied);
63             return 0;
64         }
65         key = rehash(key, h->size);    // 재해시
66         p = &h->table[key];
67     }
68     return 2;                          // 해시 테이블이 가득 참
69 }
70
```

(실습 10-7[B]에서 계속)



10-1 해시법

오픈 주소법 이해하기 - (7)

■ 실습 10-7 - (3)

Do it! 실습 10-7[B]

• 완성 파일 chap10/ClosedHash.c

```
01  /*--- 데이터 삭제 ---*/
02  int Remove(ClosedHash *h, const Member *x)
03  {
04      Bucket *p = Search(h, x);
05      if(p == NULL)
06          return 1;                // 이 키의 값은 존재하지 않음
07      p->stat = Deleted;
08      return 0;
09  }
```

```
11  /*--- 해시 테이블 덤프 ---*/
12  void Dump(const ClosedHash *h)
13  {
14      for(int i = 0; i < h->size; i++) {
15          printf("%02d: ", i);
16          switch(h->table[i].stat) {
17              case Occupied: printf("%d(%)s\n",
18                                  h->table[i].data.no, h->table[i].data.name); break;
19              case Empty: printf("-- 미등록 --\n"); break;
20              case Deleted: printf("-- 삭제 마침 --\n"); break;
21          }
22      }
23  }

25  /*--- 모든 데이터 삭제 ---*/
26  void Clear(ClosedHash *h)
27  {
28      for(int i = 0; i < h->size; i++)    // 모든 버킷을
29          h->table[i].stat = Empty;      // 공백으로 변경
30  }
31
32  /*--- 해시 테이블 종료 ---*/
33  void Terminate(ClosedHash *h)
34  {
35      Clear(h);                        // 모든 데이터 삭제
36      free(h->table);                  // 해시 테이블 배열의 메모리 해제
37      h->size = 0;                      // 해시 테이블 크기를 클리어
38  }
```



10-1 해시법

오픈 주소법 이해하기 - (8)

■ 실습 10-8 - (1)

- 오픈 주소법으로 구현한 해시 프로그램

Do it! 실습 10-8

• 완성 파일 chap10/ClosedHashTest.c

```
01 // 오픈 주소법으로 구현한 해시 사용
02
03 #include <stdio.h>
04 #include "Member.h"
05 #include "ClosedHash.h"
06
07 /*--- 메뉴 ---*/
08 typedef enum {
09     TERMINATE, ADD, DELETE, SEARCH, CLEAR, DUMP
10 } Menu;
11
12 /*--- 메뉴 선택 ---*/
13 Menu SelectMenu(void)
14 {
15     int ch;
16     do {
17         printf("(1)추가 (2)삭제 (3)검색 (4)모두 삭제 (5)덤프 (0)종료: ");
18         scanf("%d", &ch);
19     } while(ch < TERMINATE || ch > DUMP);
20     return(Menu) ch;
21 }
```

```
23 /*--- 메인 ---*/
24 int main(void)
25 {
26     Menu menu;           // 메뉴
27     ClosedHash hash;     // 해시 테이블
28     Initialize(&hash, 13); // 해시 테이블 초기화
29     do {
30         int result;
31         Member x;
32         Bucket *temp;
33         switch(menu = SelectMenu()) {
34             case ADD:           /*--- 데이터 추가 ---*/
35                 x = ScanMember("추가", MEMBER_NO | MEMBER_NAME);
36                 result = Add(&hash, &x);
37                 if(result)
38                     printf("\a오류: 추가에 실패했습니다(%s).\n",
39                             (result == 1) ? "등록 마침: "메모리 부족");
40                 break;
41
42             case DELETE:        /*--- 데이터 삭제 ---*/
43                 x = ScanMember("삭제", MEMBER_NO);
44                 result = Remove(&hash, &x);
45                 if(result == 1)
46                     printf("\a오류: 이 번호의 데이터는 존재하지 않습니다.\n");
47                 break;
```




10-1 해시법

오픈 주소법 이해하기 - (10)

체인법 실행 결과

실행 결과(체인법)

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 5

```
00
01 → 14(최윤미) → 1(붉은꼬리) ..... 같은 해시값을 갖는 데이터가 링크되어 있습니다.
02
03
04
05 → 5(박현규)
06 ..... 해시 테이블의 내용을 출력
07
08
09
10 → 10(지나)
11
12 → 12(김영준)
```

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 2

삭제할 데이터를 입력하세요.

번호: 14 14를 삭제

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 5

```
00
01 → 1(붉은꼬리)
02
03
04
05 → 5(박현규) ..... 해시 테이블의 내용을 출력
06
07
08
09
10 → 10(지나)
11
12 → 12(김영준)
```

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 0



10-1 해시법

오픈 주소법 이해하기 - (11)

오픈 주소법 실행 결과 - (1)

실행 결과(오픈 주소법)

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 1
추가할 데이터를 입력하세요.

번호: 1 {1 붉은꼬리}를 추가
이름: 붉은꼬리

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 1
추가할 데이터를 입력하세요.

번호: 5 {5 박현규}를 추가
이름: 박현규

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 1
추가할 데이터를 입력하세요.

번호: 10 {10 지나}를 추가
이름: 지나

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 1
추가할 데이터를 입력하세요.

번호: 12 {12 김영준}를 추가
이름: 김영준

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 1
추가할 데이터를 입력하세요.

번호: 14 {14 최윤미}를 추가
이름: 최윤미

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 3
검색할 데이터를 입력하세요.

번호: 5 5를 검색
검색에 성공했습니다: 5 박현규

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 5

```
00: -- 미등록 --  
01: 1(붉은꼬리)  
02: 14(최윤미)  
03: -- 미등록 --  
04: -- 미등록 --  
05: 5(박현규)  
06: -- 미등록 --  
07: -- 미등록 --  
08: -- 미등록 --  
09: -- 미등록 --  
10: 10(지나)  
11: -- 미등록 --  
12: 12(김영준)
```

해시 테이블의 내용을 출력



10-1 해시법

오픈 주소법 이해하기 - (12)

오픈 주소법 실행 결과 - (2)

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 2

삭제할 데이터를 입력하세요.

번호: 14

14를 삭제

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 5

00: -- 미등록 --
01: 1(붉은꼬리)
02: -- 삭제 마침 --
03: -- 미등록 --
04: -- 미등록 --
05: 5(박현규)
06: -- 미등록 --
07: -- 미등록 --
08: -- 미등록 --
09: -- 미등록 --
10: 10(지나)
11: -- 미등록 --
12: 12(김영준)

해시 테이블의 내용을 출력

(1) 추가 (2) 삭제 (3) 검색 (4) 모두 삭제 (5) 덤프 (0) 종료: 0